# TicketSec Arm64 — Security Ticket Classifier Optimized for Cloud AI

> **Arm Create: AI Optimization Challenge 2026** — Cloud AI Track
> An open-source template that optimizes a real-world security ticket classification model for Arm64 cloud inference, delivering measurable improvements in model size, latency, and throughput while maintaining accuracy.

---

## Quick Start (3 commands)

```
# 1. Clone and enter
git clone https://github.com/YOUR_USERNAME/ticketsec-arm64.git
cd ticketsec-arm64

# 2. Install dependencies
pip install -r requirements.txt

# 3. Train, convert, quantize, and benchmark
python train.py && python convert_onnx.py && python quantize.py && python benchmark.py

# 4. Start the API
python api.py
# Open http://localhost:8000/docs
```

---

## What This Project Does

Security operations centers (SOCs) process thousands of support tickets daily. Many contain potential security incidents (phishing, malware, unauthorized access). Manual triage is slow and error-prone. This project provides an **AI-native, Arm64-optimized classifier** that:

- Categorizes tickets into 6 security classes: `Phishing`, `Malware`, `Unauthorized_Access`, `Data_Breach`, `DDoS`, `False_Positive`
- Runs inference **faster** on Arm64 cloud instances (AWS Graviton / Oracle Ampere)
- Reduces model size via **INT8 dynamic quantization**
- Ships as a **reusable open-source template** with Docker multi-arch support

---

## Architecture

```
Security Ticket (text) → TF-IDF Vectorizer (Python) → Random Forest Classifier
                                        ↓
                                ONNX Conversion (classifier only)
                                        ↓
                            INT8 Dynamic Quantization
                                        ↓
                        ONNX Runtime + Arm Compute Library (ACL) on Arm64
                                        ↓
```

```
                    FastAPI Endpoint
```

- **Baseline:** scikit-learn RandomForest + TF-IDF (pickle)
- **Optimized:** ONNX Runtime with INT8 quantized classifier, running natively on Arm64
- **Design choice:** TF-IDF stays in Python (fast, no locale issues); ONNX handles only the heavy inference

---

## Optimization Details

| Metric | Baseline (sklearn) | ONNX INT8 | Improvement |
|---|---|---|---|
| Model Size | ~45 MB | ~8–12 MB | ↓ **~75–80%** |
| Avg Latency | ~X ms | ~Y ms | ↓ **varies by hardware** |
| Throughput | ~X req/s | ~Y req/s | ↑ **varies by hardware** |
| Accuracy | ~0.94 | ~0.94 | **± 0%** |

> *Note: Exact numbers depend on the target Arm64 hardware. Run* `benchmark.py` *on your instance to reproduce. The numbers above are illustrative — your actual benchmarks will populate the table.*

### Techniques Applied

1. **ONNX Conversion** (`skl2onnx`): RandomForest classifier exported to ONNX for cross-platform inference.
2. **Dynamic INT8 Quantization** (`onnxruntime.quantization`): Tree weights compressed to 8-bit integers with negligible accuracy loss.
3. **Arm64-Specific Runtime**: ONNX Runtime on Arm64 leverages the **Arm Compute Library (ACL)** backend for optimized inference.
4. **Docker Multi-Arch**: `linux/arm64` native image eliminates x86 emulation overhead.

---

## Project Structure

```
ticketsec-arm64/
├── train.py              # Generate synthetic dataset & train baseline model
├── convert_onnx.py       # Convert RandomForest classifier → ONNX
├── quantize.py           # Apply INT8 dynamic quantization to ONNX classifier
├── benchmark.py          # Compare baseline vs ONNX vs quantized (metrics + charts)
├── api.py                # FastAPI inference server
├── requirements.txt      # Python dependencies
├── Dockerfile            # Multi-arch Arm64 container
├── docker-compose.yml    # One-command deploy
├── run_all.sh            # Full pipeline script
├── models/               # Generated models (baseline.pkl, vectorizer.pkl, classifier.onnx,
↪  classifier_quantized.onnx)
├── data/                 # Synthetic ticket dataset (tickets.csv)
```

```
└── results/              # Benchmark JSON + charts
```

---

## Setup on AWS Graviton (t4g.micro)

### 1. Launch Instance

- AMI: Ubuntu Server 22.04 LTS (Arm64)
- Instance: `t4g.micro` (free tier eligible)
- Security Group: allow SSH (22) and HTTP (8000)

### 2. Connect & Install

```
ssh -i your-key.pem ubuntu@YOUR_EC2_IP
sudo apt update && sudo apt install -y python3-pip docker.io
pip3 install -r requirements.txt
```

### 3. Run Full Pipeline

```
python3 train.py
python3 convert_onnx.py
python3 quantize.py
python3 benchmark.py
```

### 4. Start API

```
python3 api.py
# Or with Docker:
docker-compose up --build
```

### 5. Test

```
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"text":"user reported suspicious email claiming to be from bank"}'
```

Response:

```
{
  "text": "user reported suspicious email claiming to be from bank",
  "category": "Phishing",
  "confidence": 0.92,
  "latency_ms": 18.5
}
```

---

## Benchmark Reproduction

```
python benchmark.py
```

Outputs:

- `results/benchmark_results.json` — raw metrics
- `results/benchmark_chart.png` — visual comparison

Run on **both** x86 (baseline) and Arm64 (optimized) to generate the before/after comparison.

---

## Docker (Arm64 Native)

```
# Build for Arm64
docker buildx build --platform linux/arm64 -t ticketsec-arm64 .

# Run
docker run -p 8000:8000 ticketsec-arm64

# Or simply:
docker-compose up --build
```

---

## License

MIT License — see LICENSE file.
Open source, reusable, and ready for production SOC/ITSM integrations.

---

## Contact & Credits

Built by **PhantomCrust INACIO** for the **Arm Create: AI Optimization Challenge 2026** — Cloud AI Track.
Questions? Open an issue on GitHub.